

Matériel Design

Table des matières

Fichier du thème	2
Palette de couleurs	3
Modifier la palette de couleurs	4
Ajouter les couleurs à l'application	5
Forme	6
Typographie	7
L'échelle de type <i>Material Design</i>	7
Barre supérieure	10
Version finale de l'exemple	12
Exercices	16
Références	16

Material Design est un système de conception créé par *Google* pour créer des expériences numériques de haute qualité pour *Android*, ainsi que pour d'autres plates-formes mobiles et Web. Il s'inspire du monde physique et de ses textures, y compris de la façon dont les objets reflètent la lumière et projettent des ombres. Vous y trouverez des consignes vous expliquant comment élaborer une application à l'interface lisible, attrayante et cohérente.

Dans ce chapitre, vous allez apprendre à utiliser la thématisation *Material*, qui vous permet d'utiliser *Material Design* dans votre application, et découvrirez comment personnaliser les couleurs, la typographie et les formes. Vous pouvez personnaliser votre application autant que vous le souhaitez. Vous allez également apprendre à ajouter une barre d'application supérieure pour afficher le nom et l'icône de l'application.

Fichier du thème

Dans un premier temps, il est nécessaire de configurer un projet dans *Android Studio* avec un modèle *Empty Activity*. Nommez ce projet **MaListe1**. Puis, ajouter les 2 composants: **MaListe()** et **UnePersonne()**.

```
1 @Composable
2 fun MaListe(modifier: Modifier = Modifier) {
3     Column {
4         UnePersonne("Gill", "Stephane", modifier)
5         UnePersonne("Bellier", "Nathalie", modifier)
6         UnePersonne("Atroun", "Mohand", modifier)
7     }
8 }
9
10 @Composable
11 fun UnePersonne(nom: String, prenom: String, modifier: Modifier = Modifier){
12     Card(modifier = modifier.padding(8.dp).fillMaxWidth())
13     {
14         Column {
15             Text(
16                 text = nom,
17                 modifier = Modifier.padding(4.dp)
18             )
19             Text(
20                 text = prenom,
21                 modifier = Modifier.padding(4.dp)
22             )
23         }
24     }
25 }
```

Le fichier **Theme.kt** contient toutes les informations sur le thème de l'application, lequel est défini par des couleurs, des formes et une typographie. Ce fichier se trouve dans le répertoire **ui.theme**. Dans ce fichier se trouve le composable **MaListe1Theme()**, qui définit les couleurs, la typographie et les formes de l'application.

```

1 @Composable
2 fun MaListe1Theme(
3     darkTheme: Boolean = isSystemInDarkTheme(),
4     // Dynamic color is available on Android 12+
5     dynamicColor: Boolean = true,
6     content: @Composable () -> Unit
7 ) {
8 ...
9 ...
10 ...

```

Attention ! : fichier **Theme.kt** ligne 5 mettre à false pour modifier les couleurs

Dans **MainActivity.kt**, le **MaterialTheme()** (ligne 5) est ajouté pour fournir la thématisation Material.

```

1 class MainActivity : ComponentActivity() {
2     override fun onCreate(savedInstanceState: Bundle?) {
3         super.onCreate(savedInstanceState)
4         setContent {
5             MaListe1Theme {
6                 Surface(
7                     modifier = Modifier.fillMaxSize(),
8                     color = MaterialTheme.colorScheme.background
9                 ) {
10                    MaListe()
11                }
12            }
13        }
14    }
15 }

```

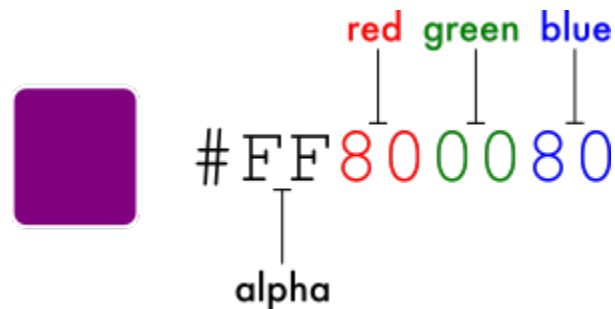
Palette de couleurs

Une palette de couleurs est la combinaison de couleurs utilisée par votre application. Les différentes combinaisons de couleurs évoquent différentes humeurs, ce qui a une influence sur la façon dont les utilisateurs se servent de votre application. Puisque nous avons travaillé avec un graphiste pour cette application, il a choisi la palette de couleurs pour nous. Toutefois, le site Web de *Material Design* fournit des conseils sur le système de couleurs. Vous pourrez ainsi en savoir plus sur les palettes de couleurs et découvrir comment en générer une vous-même.

Lorsque vous avez ajouté une couleur dans votre application *Android*, vous avez utilisé **Color.RED** ou **Color.PINK** par exemple. Comment faire si vous souhaitez ajouter une couleur plus spécifique, comme le bleu céruleen ou le vert jade ?

Dans le système *Android*, la couleur est représentée par une valeur de couleur hexadécimale (hex). Un code couleur hexadécimal commence par le caractère dièse (#), suivi de six lettres et/ou chiffres représentant les composants rouge, vert et bleu (RVB) de cette couleur. Les deux premières

lettres/chiffres font référence au rouge, les deux caractères suivants au vert et les deux derniers au bleu.



Une couleur peut également inclure une valeur alpha (lettres et/ou chiffres) qui définit la transparence de la couleur (#00 correspond à une opacité de 0 %, entièrement transparent ; #FF à une opacité de 100 %, entièrement opaque). Lorsqu'elle est incluse, la valeur alpha correspond aux deux premiers caractères du code hexadécimal. Si aucune valeur alpha n'est incluse, il s'agit par défaut de #FF, qui représente une opacité de 100 % (entièrement opaque).

Modifier la palette de couleurs

1. Ouvrez le fichier **Color.kt**. Ce fichier permet d'ajouter des couleurs à la palette de couleurs de votre application. Nous avons déjà ajouté plusieurs couleurs à la palette de couleurs par défaut.

```
1 // Mes couleurs
2 val White = Color(0xFFFFFFFF)
3 val Grey50 = Color(0xFFF8F9FA)
4 val Grey100 = Color(0xFFF1F3F4)
5 val Grey200 = Color(0xFFEEEEEE)
6 val Grey900 = Color(0xFF202124)
7 val Grey700 = Color(0xFF5F6368)
```

2. Ouvrez le fichier **Theme.kt** et remplacez **LightColorPalette** et **DarkColorScheme** par le code ci-dessous, en ajoutant les couleurs à leurs encoches.

```

1 private val DarkColorScheme = darkColorScheme(
2     background = Grey900,
3     surface = Grey700,
4     onSurface = White,
5     primary = Grey900,
6     onPrimary = White,
7     secondary = Grey100
8 )
9
10 private val LightColorScheme = lightColorScheme(
11     background = Grey200,
12     surface = Grey50,
13     onSurface = Grey900,
14     primary = Grey50,
15     onPrimary = Grey900,
16     secondary = Grey700
17 )

```

Ces encoches permettent d'harmoniser le système de conception, dans lequel les composants associés ont la même couleur.

Certains composants sont automatiquement mappés avec des encoches de couleur. Par exemple, la couleur d'arrière-plan est automatiquement mappée sur l'encoche Surface pour les composables **Surface**. Ainsi, vous n'avez pas besoin d'attribuer explicitement une couleur de surface au composable **Surface**. Il s'affichera automatiquement avec cette couleur lorsque vous définirez le thème de couleur dans votre application.

Voici la description des principales encoche (*slot*):

- La couleur **primary** est la couleur qui s'affiche le plus fréquemment sur les écrans et les composants de votre application.
- La couleur **secondary** permet d'accentuer votre application de plusieurs façons.
- La couleur de **surface** affecte les surfaces des composants, comme les vignettes, les feuilles et les menus.
- La couleur de **background** apparaît derrière le contenu déroulant.
- Les éléments de couleur **on** apparaissent **au-dessus** des autres couleurs de la palette et sont principalement appliqués au texte, à l'iconographie et aux traits. Dans notre palette de couleurs, nous avons une couleur **onSurface**, qui apparaît au-dessus de la couleur de **surface**, et une couleur **onPrimary**, qui apparaît au-dessus de la couleur **principale**.

Ajouter les couleurs à l'application

Maintenant que les couleurs ont été ajoutées, vous pouvez les utiliser dans différents espaces de l'application. Vous utiliserez :

- La couleur **background** pour l'arrière-plan de la liste (ligne 4).
- La couleur **surface** pour l'arrière-plan de l'élément de la liste, pour bien le distinguer par

rapport à l'arrière-plan de la liste (ligne 15).

- La couleur `onSurface` pour le texte, pour une lisibilité optimale par rapport à la couleur de l'élément de la liste (ligne 21 et 26).

```
1 @Composable
2 fun MaListe(modifier: Modifier = Modifier) {
3     Column(modifier =
4         Modifier.background(MaterialTheme.colorScheme.background)){
5         UnePersonne("Gill", "Stephane", modifier)
6         UnePersonne("Bellier", "Nathalie", modifier)
7         UnePersonne("Atroun", "Mohand", modifier)
8     }
9 }
10
11 @Composable
12 fun UnePersonne(nom: String, prenom: String, modifier: Modifier = Modifier){
13     Card(modifier = modifier.padding(8.dp).fillMaxWidth(),
14         colors = CardDefaults.cardColors(
15             containerColor = MaterialTheme.colorScheme.surface,
16         )
17     ) {
18         Column {
19             Text(
20                 text = nom,
21                 color = MaterialTheme.colorScheme.onSurface,
22                 modifier = Modifier.padding(4.dp)
23             )
24             Text(
25                 text = prenom,
26                 color = MaterialTheme.colorScheme.onSurface,
27                 modifier = Modifier.padding(4.dp)
28             )
29         }
30     }
31 }
```

Forme

Par défaut, toutes les formes de votre application sont rectangulaires. Toutefois, l'application d'une forme peut modifier considérablement l'apparence d'un composable. Les formes permettent de capter l'attention, d'identifier les composants, de communiquer un état et d'exprimer une image de marque.

De nombreuses formes sont définies à l'aide de `RoundedCornerShape`, qui décrit un rectangle avec des angles arrondis. Le nombre transmis définit l'arrondi des angles. Si vous utilisez `RoundedCornerShape(percent = 0)`, alors le rectangle n'a pas d'angles arrondis. Si vous utilisez `RoundedCornerShape(percent = 50)`, les angles sont entièrement arrondis. Si vous souhaitez tester des formes plus complexes, le site de *Material* propose un outil de personnalisation des formes.

Le fichier **Shape.kt** est utilisé pour définir les formes des composants dans *Compose*. Il existe trois types de composants : petits, moyens et grands. Dans cette section, vous allez modifier le composant **Card**, qui est défini sur la taille **medium**. Les composants sont regroupés en catégories de formes en fonction de leur taille.

```
1 package com.example.maliste1.ui.theme
2
3 import androidx.compose.foundation.shape.RoundedCornerShape
4 import androidx.compose.material3.Shapes
5 import androidx.compose.ui.unit.dp
6
7 val Shapes = Shapes(
8     small = RoundedCornerShape(4.dp),
9     medium = RoundedCornerShape(16.dp),
10    large = RoundedCornerShape(0.dp)
11 )
```

et ajouter dans le fichier **Theme.kt** ligne 4.

```
1 MaterialTheme(
2     colorScheme = colorScheme,
3     typography = Typography,
4     shapes = Shapes,
5     content = content
6 )
```

Typographie

L'échelle de type *Material Design*

Une échelle de type est une sélection de styles de police pouvant être utilisés dans une application pour garantir un style flexible mais cohérent. L'échelle de type *Material Design* comprend 13 styles de police compatibles avec le système de types. N'utilisez ces choix que si vous souhaitez personnaliser votre application. Si vous ne savez pas quoi définir pour les catégories d'échelle de type, vous pouvez utiliser une échelle de typographie par défaut.

M3 Typescale

Display Large	Roboto 57/64 . 0
Display Medium	Roboto 45/52 . 0
Display Small	Roboto 36/44 . 0
Headline Large	Roboto 32/40 . 0
Headline Medium	Roboto 28/36 . 0
Headline Small	Roboto 24/32 . 0
Title Large	NEW - Roboto Medium 22/28 . 0
Title Medium	Roboto Medium 16/24 . +0.15
Title Small	Roboto Medium 14/20 . +0.1
Body Large	Roboto 16/24 . +0.5
Body Medium	Roboto 14/20 . +0.25
Body Small	Roboto 12/16 . +0.4
Label Large	Roboto Medium 14/20 . +0.1
Label Medium	Roboto Medium 12/16 . +0.5
Label Small	NEW - Roboto Medium 11/16 . +0.5

Material Design 2

M2 Typescale

Headline 1	H1/Roboto/Light/96px
Headline 2	H2/Roboto/Light/60px
Headline 3	H3/Roboto/Regular/48px
Headline 4	H4/Roboto/Regular/34px
Headline 5	H5/Roboto/Regular/24px
Headline 6	H6/Roboto/Medium/20px
Subtitle 1	Subtitle 1/Roboto/Regular/16px
Subtitle 2	Subtitle 2/Roboto/Medium/14px
Body 1	Body 1/Roboto/Regular/16px
Body 2	Body 2/Roboto/Regular/14px
Caption	Caption/Roboto/Regular/12px
Button	BUTTON/ROBOTO/MEDIUM/14PX
Overline	OVERLINE/ROBOTO/REGULAR/10PX

L'échelle de type contient des catégories de texte réutilisables, dont chacune a une utilisation et une signification spécifiques. Dans notre application, nous n'utilisons que les catégories "Titre" et "Corps".

Polices

La plate-forme Android propose quelques polices, mais vous pouvez utiliser une police non fournie par défaut pour personnaliser votre application. Les polices personnalisées apportent une touche personnelle et peuvent être utilisées pour le branding.

Des police de caractères sont disponible sur le site: <https://fonts.google.com/>. Une fois les police télécharger, les décompresser et les ajouter dans le répertoire **res/font**. Il est nécessaire, d'importer les police des caractères en modifiant le fichier **Type.kt**

```

1 // Ajout d'une police
2 val AbrilFatface = FontFamily(
3     Font(R.font.abril_fatface_regular)
4 )
5
6
7 val Typography = Typography(
8     bodyLarge = TextStyle(
9         fontFamily = FontFamily.Default,
10        fontWeight = FontWeight.Normal,
11        fontSize = 16.sp,
12        lineHeight = 24.sp,
13        letterSpacing = 0.5.sp
14    ),
15    displayMedium = TextStyle(
16        fontFamily = FontFamily.Default,
17        fontWeight = FontWeight.Bold,
18        fontSize = 20.sp
19    ),
20
21    // Utilisation de la nouvelle police
22    titleLarge = TextStyle(
23        fontFamily = AbrilFatface,
24        fontWeight = FontWeight.Normal,
25        fontSize = 22.sp
26    )
27 )
28 ...

```

Modifier la taille des polices de caractères

```

1 ...
2 Column {
3     Text(
4         text = nom,
5         color = MaterialTheme.colorScheme.onSurface,
6         style = MaterialTheme.typography.displaySmall,
7         modifier = Modifier.padding(4.dp)
8     )
9     Text(
10        text = prenom,
11        color = MaterialTheme.colorScheme.onSurface,
12        style = MaterialTheme.typography.bodyLarge,
13        modifier = Modifier.padding(4.dp)
14    )
15 }
16 ...

```

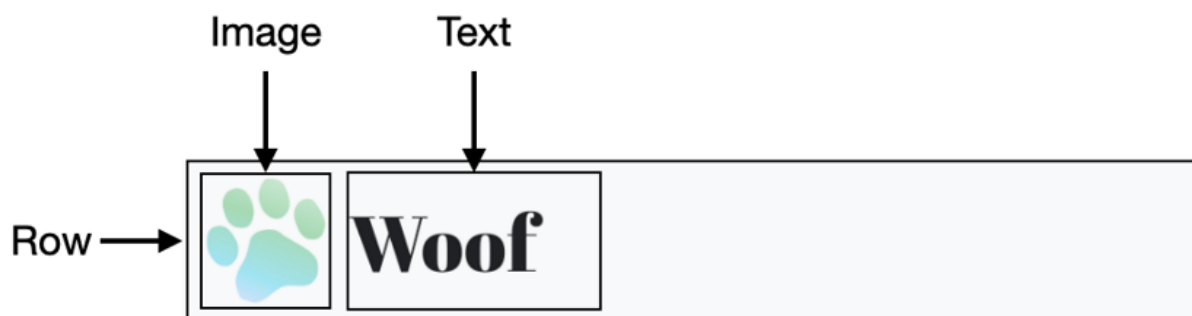
Ligne 6 et 12 la taille du texte affiché est modifié

Barre supérieure

Un **Scaffold**, ou échafaudage, est une mise en page qui fournit des emplacements pour divers composants et éléments sur l'écran, tels que **Image**, **Row** ou **Column**. Un **Scaffold** fournit également un emplacement pour **topBar**, que vous utiliserez dans cette section.

La **topBar** peut être utilisée de plusieurs façons, mais dans ce cas, elle vous permettra de créer une image de marque et de donner une touche personnelle à votre application. Vous allez créer un composable qui ressemble à la capture d'écran ci-dessous, et l'insérer dans la section **topBar** d'un **Scaffold**.

Dans cette application, la barre supérieure se compose d'une ligne incluant un logo et un nom d'appli. Le logo inclut une petite patte en dégradé de couleur, et le titre de l'application.



1. Créer la barre supérieur

```
1 @Composable
2 fun BarreSup(modifier: Modifier = Modifier) {
3     Row( modifier = modifier
4         .fillMaxWidth()
5         .background(color = MaterialTheme.colorScheme.surface),
6         verticalAlignment = Alignment.CenterVertically
7     ) {
8         Image(
9             modifier = Modifier
10                .size(72.dp)
11                .padding(4.dp),
12             painter = painterResource(R.drawable._3d),
13             contentDescription = "neige"
14         )
15         Text(
16             text = "mon Carnet",
17             style = MaterialTheme.typography.displaySmall,
18         )
19     }
20 }
21 }
22 }
```

2. Intégrer la barre supérieur à l'application

```
1 @Composable
2 fun MaListe(modifier: Modifier = Modifier) {
3     Scaffold(
4         topBar = {
5             BarreSup()
6         },
7     ){ innerPadding -> // padding calculated by scaffold
8         Column (modifier =
9             Modifier.background(MaterialTheme.colorScheme.background)
10            .padding(paddingValues = innerPadding)
11        ){
12            UnePersonne("Gill", "Stephane", modifier)
13            UnePersonne("Bellier", "Nathalie", modifier)
14            UnePersonne("Atroun", "Mohand", modifier)
15        }
16    }
17 }
```

L'ajout de la zone de texte pourrait générer une erreur. Si le message d'erreur suivant apparaît: *This material API is experimental and is likely to change or to be removed in the future.*, il est nécessaire d'ajouter les instructions suivantes dans le fichier **build.gradle**

ATTENTION!

```
1 kotlinOptions {
2     allWarningsAsErrors = false
3     freeCompilerArgs += [
4         '-opt-
5         in=androidx.compose.material3.ExperimentalMaterial3Api'
6     ]
7 }
```

Le message d'erreur devrait disparaître après avoir synchronisé le *Gradle* ou après avoir redémarrer Android Studio.

Source: <https://stackoverflow.com/questions/71069737/this-material-api-is-experimental-and-is-likely-to-change-or-to-be-removed-in-th>

Message d'erreur: Duplicate class kotlin.collections.jdk8.CollectionsJDK8Kt found in modules kotlin-stdlib-1.8.10...

ATTENTION!

```
1 constraints {
2     implementation("org.jetbrains.kotlin:kotlin-stdlib-
jdk7:1.8.0") {
3         because("kotlin-stdlib-jdk7 is now a part of
kotlin-stdlib")
4     }
5     implementation("org.jetbrains.kotlin:kotlin-stdlib-
jdk8:1.8.0") {
6         because("kotlin-stdlib-jdk8 is now a part of
kotlin-stdlib")
7     }
8 }
```

Le message d'erreur devrait disparaître après avoir synchronisé le *Gradle* ou après avoir redémarrer Android Studio.

Source: <https://stackoverflow.com/questions/71069737/this-material-api-is-experimental-and-is-likely-to-change-or-to-be-removed-in-th>

Version finale de l'exemple

```
1 package com.example.maliste1
2
3 import android.annotation.SuppressLint
4 import android.os.Bundle
5 import androidx.activity.ComponentActivity
6 import androidx.activity.compose.setContent
7 import androidx.compose.foundation.background
8 import androidx.compose.foundation.Image
9 import androidx.compose.foundation.layout.Column
10 import androidx.compose.foundation.layout.Row
11 import androidx.compose.foundation.layout.fillMaxSize
12 import androidx.compose.foundation.layout.fillMaxWidth
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.layout.size
15 import androidx.compose.material3.Card
16 import androidx.compose.material3.CardDefaults
17 import androidx.compose.material3.MaterialTheme
18 import androidx.compose.material3.Scaffold
19 import androidx.compose.material3.Surface
20 import androidx.compose.material3.Text
21 import androidx.compose.runtime.Composable
22 import androidx.compose.ui.Alignment
23 import androidx.compose.ui.Modifier
24 import androidx.compose.ui.res.painterResource
25 import androidx.compose.ui.semantics.Role.Companion.Image
```

```

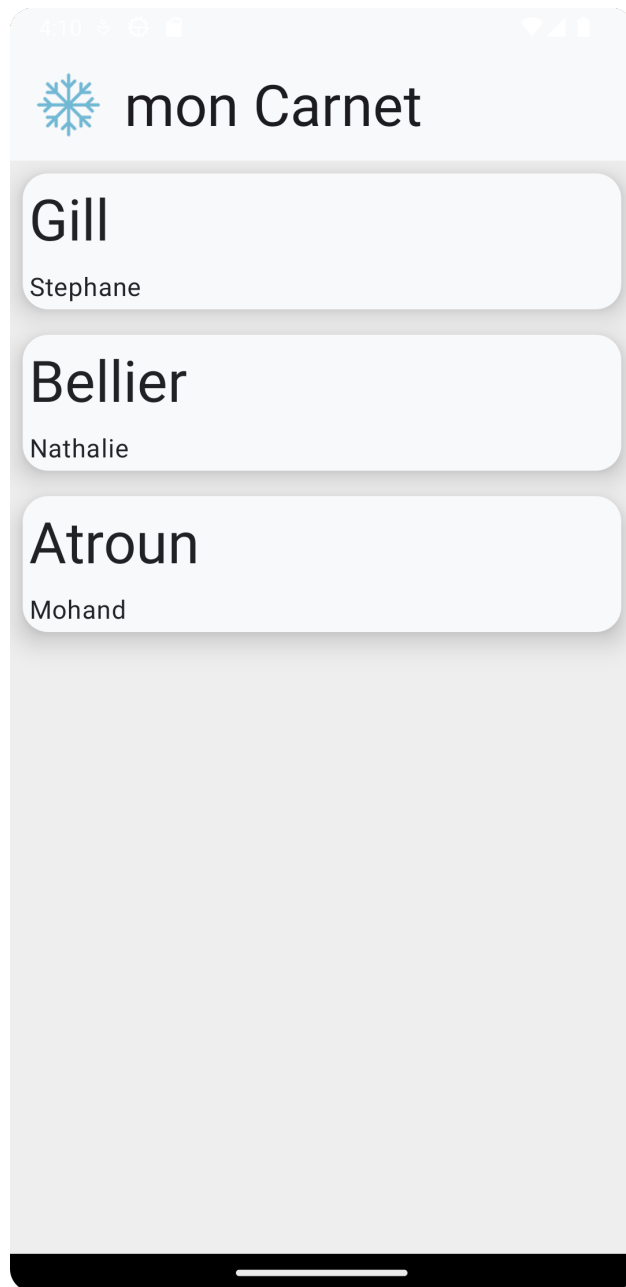
26 import androidx.compose.ui.tooling.preview.Preview
27 import androidx.compose.ui.unit.dp
28 import com.example.maliste1.ui.theme.Maliste1Theme
29
30 class MainActivity : ComponentActivity() {
31     override fun onCreate(savedInstanceState: Bundle?) {
32         super.onCreate(savedInstanceState)
33         setContent {
34             Maliste1Theme {
35                 Surface(
36                     modifier = Modifier.fillMaxSize(),
37                     color = MaterialTheme.colorScheme.background
38                 ) {
39                     Maliste()
40                 }
41             }
42         }
43     }
44 }
45
46 @Composable
47 fun Maliste(modifier: Modifier = Modifier) {
48     Scaffold(
49         topBar = {
50             BarreSup()
51         },
52     ){ innerPadding -> // padding calculated by scaffold
53         Column(modifier =
54             Modifier.background(MaterialTheme.colorScheme.background)
55                 .padding(paddingValues = innerPadding)
56         ){
57             UnePersonne("Gill", "Stephane", modifier)
58             UnePersonne("Bellier", "Nathalie", modifier)
59             UnePersonne("Atroun", "Mohand", modifier)
60         }
61     }
62 }
63
64 @Composable
65 fun UnePersonne(nom: String, prenom: String, modifier: Modifier = Modifier){
66     Card(modifier = modifier.padding(8.dp).fillMaxWidth(),
67         colors = CardDefaults.cardColors(
68             containerColor = MaterialTheme.colorScheme.surface,
69         ),
70         elevation = CardDefaults.cardElevation(
71             defaultElevation = 10.dp
72         )
73     ) {
74         Column {
75             Text(
76                 text = nom,
77                 color = MaterialTheme.colorScheme.onSurface,
78                 style = MaterialTheme.typography.displaySmall,
79                 modifier = Modifier.padding(4.dp)
80             )

```

```

81         Text(
82             text = prenom,
83             color = MaterialTheme.colorScheme.onSurface,
84             style = MaterialTheme.typography.bodyLarge,
85             modifier = Modifier.padding(4.dp)
86         )
87     }
88 }
89 }
90
91 @Composable
92 fun BarreSup(modifier: Modifier = Modifier) {
93     Row( modifier = modifier
94         .fillMaxWidth()
95         .background(color = MaterialTheme.colorScheme.surface),
96         verticalAlignment = Alignment.CenterVertically
97     ) {
98         Image(
99             modifier = Modifier
100                 .size(72.dp)
101                 .padding(4.dp),
102             painter = painterResource(R.drawable._3d),
103             contentDescription = "neige"
104         )
105         Text(
106             text = "mon Carnet",
107             style = MaterialTheme.typography.displaySmall,
108         )
109     }
110 }
111 }
112 }
113
114 @Preview(showBackground = true)
115 @Composable
116 fun GreetingPreview() {
117     MaListe1Theme (darkTheme = false){
118         MaListe()
119     }
120 }

```



Exercices

Exercice 1

Refaire l'exemple des notes de cours

Références

- [Concevoir des applications attrayantes](#)
- [Material Design](#)